



UNIVERSIDAD TECNOLÓGICA DE QUERÉTARO

Nombre de la Organización:

UNIVERSIDAD TECNOLÓGICA DE QUERÉTARO (UTEQ)

Presenta:

VICTOR GUADALUPE LOPEZ CEBREROS

Grupo:

IDGS17

Práctica 2 - Parte 2

1. Alcance y uso permitido

Este reporte documenta un ejercicio **exclusivamente educativo**. Todas las pruebas descritas se ejecutaron sobre la aplicación local VulnerableApp, construida por el propio estudiante en un entorno de laboratorio aislado, sin exposición a red pública. No se probó ninguno de estos procedimientos sobre aplicaciones, servicios, redes o cuentas reales de terceros. El objetivo del ejercicio es comprender el mecanismo de las vulnerabilidades para poder **prevenir las y remediarlas** en desarrollos propios.

2. Objetivo general

Reproducir de forma controlada un conjunto de vulnerabilidades del OWASP Top 10 en una aplicación ASP.NET Core MVC, analizar su impacto técnico y de negocio, y aplicar las mitigaciones correspondientes, comparando el comportamiento de la aplicación antes y después de la corrección.

3. Entorno de trabajo

Elemento	Versión / detalle
Sistema operativo	<i>(Windows 11 / macOS / Linux)</i>
SDK	<i>.NET (8.0 / 9.0)</i>
Framework	ASP.NET Core MVC
ORM	Entity Framework Core
Base de datos	<i>(SQL Server LocalDB / SQLite)</i>
Editor	Visual Studio Code
Cliente HTTP	<i>(Postman / navegador / Thunder Client)</i>
Navegador	<i>(Chrome / Edge) + DevTools</i>
Control de versiones Git — rama main (vulnerable) y rama secure (remediada)	

```
info: Microsoft.Hosting.Lifetime[14]
      Now listening on: http://localhost:5277
info: Microsoft.Hosting.Lifetime[0]
      Application started. Press Ctrl+C to shut down.
info: Microsoft.Hosting.Lifetime[0]
      Hosting environment: Development
info: Microsoft.Hosting.Lifetime[0]
      Content root path: C:\Users\victo\Documents\practicas\VulnerableApp
```

PARTE I — SEGG-U1-P2D: Cross-Site Scripting (XSS)

4.1 Objetivo de la práctica

Comprender cómo la ausencia de validación de entrada y, sobre todo, de **codificación de salida**, permite que contenido no confiable enviado por un usuario sea interpretado y ejecutado por el navegador de otros usuarios.

4.2 Implementación del módulo vulnerable

Se creó el controlador CommentController, que almacena los comentarios en una lista estática en memoria, y la vista Views/Comment/Index.cshtml, que los renderiza.

```
CommentController.cs X
VulnerableApp > Controllers > CommentController.cs > ...
You, 3 days ago | 1 author (You)
1 using Microsoft.AspNetCore.Mvc;
2
3 You, 3 days ago | 1 author (You)
4 namespace VulnerableApp.Controllers
5 {
6     0 references | You, 3 days ago | 1 author (You)
7     public class CommentController : Controller
8     {
9         2 references
10        private static List<string> _comments = new();
11
12        0 references
13        public IActionResult Index()
14        {
15            return View(_comments);
16        }
17
18        [HttpPost]
19        0 references
20        public IActionResult AddComment(string comment)
21        {
22            if (!string.IsNullOrEmpty(comment))
23            {
24                _comments.Add(comment);
25            }
26            return RedirectToAction("Index");
27        }
28    }
29 }
```

Texto de análisis del código (redactado para el reporte):

El punto exacto de la falla se encuentra en la vista, no en el controlador. Razor codifica automáticamente en HTML cualquier valor emitido con la sintaxis `@variable`; sin embargo, `@Html.Raw()` **desactiva explícitamente esa protección** e inserta la cadena tal cual en el árbol DOM. El controlador agrava el problema al no aplicar ninguna validación ni saneamiento sobre el parámetro `comment` antes de persistirlo: la única verificación existente (`!string.IsNullOrEmpty`) comprueba presencia, no contenido.

Adicionalmente, dado que los comentarios se almacenan en una colección compartida por todos los usuarios de la aplicación, la vulnerabilidad corresponde a un **XSS almacenado (stored/persistent)**, la variante de mayor severidad: el payload no requiere que la víctima haga clic en un enlace preparado, sino que se ejecuta automáticamente en cada visita a la página de comentarios. Al ser una lista static en memoria, los datos se pierden al reiniciar la aplicación, lo cual es una limitación del laboratorio y no una medida de seguridad.

4.3 Prueba con comentario legítimo

Se ingresó un comentario de texto plano para verificar el funcionamiento normal del módulo.

4.4 Prueba con contenido no confiable

Se ingresó, únicamente en la aplicación local de laboratorio, el siguiente contenido:

```
<script>alert('XSS')</script>
```

4.5 Análisis del riesgo

¿Por qué el navegador interpreta el contenido? El navegador no distingue el origen ni la intención del HTML que recibe: analiza el documento de forma secuencial y ejecuta todo lo que encuentre dentro de una etiqueta `<script>`, porque para él ese código es indistinguible del código legítimo que el servidor pretendía enviar. Al usar `@Html.Raw`, el servidor entrega los caracteres `<`, `>` y `/` sin transformar, de modo que dejan de ser datos y pasan a formar parte de la estructura del documento. El código inyectado se ejecuta bajo el **origen** de la aplicación, heredando por completo su contexto de confianza.

¿Qué información podría estar en riesgo?

- **Cookies de sesión** accesibles desde `document.cookie` si no tienen el atributo `HttpOnly`, lo que permite el secuestro de sesión.

- **Tokens** almacenados en localStorage o sessionStorage, siempre accesibles desde JavaScript.
- **Datos mostrados en pantalla** al usuario víctima (saldos, correos, información personal), que pueden ser leídos del DOM y enviados a un servidor externo.
- **Credenciales**, mediante formularios falsos inyectados en la página legítima (phishing con dominio y certificado válidos).
- **Acciones ejecutadas en nombre de la víctima**: el script puede emitir peticiones autenticadas con las cookies del usuario, alterando o eliminando datos.

Relación con validación de entrada y codificación de salida: Ambos controles son necesarios pero cumplen funciones distintas. La **validación de entrada** debe aplicarse lo más cerca posible del punto donde el dato entra al sistema, preferentemente con listas de permitidos (formato, longitud, conjunto de caracteres esperado); es una defensa útil, pero insuficiente por sí sola, ya que existen contextos legítimos donde caracteres especiales deben aceptarse. La **codificación de salida** es la defensa primaria contra XSS y debe aplicarse en el momento del renderizado y en función del contexto de destino (cuerpo HTML, atributo, JavaScript, URL o CSS), porque cada contexto tiene reglas de escape diferentes. Como defensa en profundidad se suman una política **Content-Security-Policy** que restrinja el origen de los scripts y prohíba los scripts en línea, y cookies con HttpOnly, Secure y SameSite.

4.6 Evidencias solicitadas (P2D)

# Evidencia solicitada	Archivo	
1 Comentario legítimo	evidencias/P2D/03-comentario-legitimo.png	
2 Contenido no confiable ingresado	evidencias/P2D/04-payload-ingresado.png	
3 Comportamiento en navegador	evidencias/P2D/05-alert-ejecutado.png	
4 Código vulnerable	evidencias/P2D/01-codigo-controller.png, codigo-vista.png	02-

PARTE II — SEGG-U1-P2E: Insecure Direct Object References (IDOR)

5.1 Objetivo de la práctica

Analizar los riesgos de exponer recursos internos mediante identificadores directos, sin validar autenticación, autorización ni pertenencia (*ownership*) del recurso solicitado.

5.2 Implementación del API vulnerable

Se creó ApiController con dos endpoints: GET /api/user/{id} y GET /api/users.

5.3 Consumo de endpoints

Petición	Resultado observado
GET /api/user/1	200 OK — datos completos del usuario 1
GET /api/user/2	200 OK — datos completos del usuario 2 (<i>sin haber iniciado sesión como él</i>)
GET /api/user/3	200 OK — datos completos del usuario 3
GET /api/users	200 OK — listado completo de la tabla Users

5.4 Campos expuestos en la respuesta

Campo devuelto	¿Sensible?	Justificación
Id	Bajo	Identificador interno; su exposición habilita la enumeración
Username	Medio	Dato de identificación, insumo para ataques de fuerza bruta y phishing
Email	Alto	Dato personal (PII); insumo para phishing dirigido y recuperación de cuenta
Balance	Alto	Información financiera de un tercero

Campo devuelto

¿Sensible? Justificación

Password	Crítico	Credencial en texto plano; compromiso total de la cuenta
----------	---------	--

En GET /api/users la situación es más grave: al retornar directamente la entidad User del modelo, la respuesta incluye **todas** las propiedades de la tabla, incluidos los campos que nunca debieron salir del servidor. Esto ilustra el riesgo de serializar entidades de dominio en lugar de objetos de transferencia (DTO) diseñados para exposición.

5.5 Análisis de la autorización faltante

El endpoint no realiza ninguna de las tres verificaciones que debería:

1. **Autenticación** (*¿quién eres?*): no existe atributo [Authorize] ni consulta a la sesión; cualquier cliente anónimo obtiene respuesta 200.
2. **Autorización** (*¿tienes permiso para esta operación?*): no se evalúa rol ni privilegio; un usuario común accede al mismo listado que un administrador.
3. **Ownership** (*¿este recurso es tuyo?*): el id proviene íntegramente de la URL y se usa como clave de búsqueda sin contrastarlo contra la identidad del solicitante. El servidor confía en un parámetro controlado por el atacante para decidir qué datos entregar.

Por qué los ids consecutivos facilitan la enumeración: Al ser enteros autoincrementales, el espacio de identificadores es completamente predecible: conocido un id válido, todos los demás se deducen por incremento. Un atacante puede recorrer el rango 1..N con un script trivial y extraer la base de datos completa en segundos, sin necesidad de adivinar ni de forzar credenciales. Además, el id máximo alcanzado revela información de negocio (número aproximado de usuarios registrados y ritmo de crecimiento). El uso de identificadores no secuenciales, como GUID, **no sustituye al control de autorización** —es una medida de ofuscación—, pero eleva significativamente el costo del descubrimiento automatizado y debe combinarse con verificación de pertenencia y limitación de tasa.

5.6 Clasificación del riesgo

Recurso	Acceso observado	Riesgo	Categoría OWASP Top 10 2021	Severidad
/api/user/{id}	Consulta de cualquier usuario por id	Acceso a datos de terceros	A01 — Broken Access Control	Alta
/api/users	Listado completo sin autenticación	Exposición masiva de datos	A01 — Broken Access Control	Crítica
Campo Password	Retornado en la respuesta almacenado en texto plano	Compromiso y credenciales en reutilización otros servicios	A02 — Cryptographic Failures	Crítica
Ausencia de Endpoints [Authorize]	anónimos	Falla identificación autenticación	A07 — Identification and Authentication Failures	Alta

5.7 Evidencias solicitadas (P2E)

# Evidencia solicitada	Archivo
1 GET /api/user/1	evidencias/P2E/08-get-user-1.png
2 GET /api/user/2	evidencias/P2E/09-get-user-2.png
3 GET /api/users	evidencias/P2E/11-get-users.png
4 Código vulnerable	evidencias/P2E/07-codigo-apicontroller.png
5 Tabla de campos sensibles	Sección 5.4 de este reporte

PARTE III — SEGG-U1-P2F: Remediación de Vulnerabilidades OWASP

6.1 Objetivo de la práctica

Aplicar controles de seguridad que mitiguen las vulnerabilidades identificadas en las prácticas previas y comparar el comportamiento de la aplicación antes y después de la corrección.

6.2 Estrategia de remediación

Se creó la rama `secure` a partir de la rama `vulnerable`, de manera que ambas versiones del código coexisten y permiten una comparación directa. Se mantiene la rama original sin modificar como línea base de evidencia.

```
git checkout -b secure
```

6.3 Mitigación 1 — SQL Injection

Antes: la consulta se construía por concatenación de cadenas, de modo que la entrada del usuario pasaba a formar parte de la sentencia SQL y podía alterar su estructura lógica (por ejemplo con `' OR '1'='1'`).

Después: se sustituyó por una consulta LINQ sobre el `DbSet`. `Entity Framework Core` traduce la expresión a SQL **parametrizado**, enviando el valor de búsqueda como parámetro y no como parte del texto de la sentencia. El motor de base de datos recibe la estructura de la consulta y los datos por canales separados, por lo que ningún carácter contenido en la entrada puede modificar la lógica original.

Como criterio general de diseño se evita `FromSqlRaw` y `ExecuteSqlRaw`; cuando el uso de SQL nativo sea indispensable, debe emplearse `FromSqlInterpolated` o parámetros explícitos, nunca interpolación manual de cadenas.

6.4 Mitigación 2 — Protección de contraseñas

Antes: el modelo `User` almacenaba la propiedad `Password` en texto plano, y ese mismo campo se devolvía en las respuestas del API.

Después: el campo se reemplazó por `PasswordHash` y las contraseñas se procesan con **BCrypt**, un algoritmo de hashing adaptativo diseñado específicamente para credenciales. `BCrypt` incorpora un *salt* aleatorio único por registro —lo que impide el uso de tablas precalculadas y evita que dos usuarios con la misma contraseña produzcan el mismo hash— y un factor de trabajo configurable que permite encarecer deliberadamente cada cálculo, degradando la viabilidad de los ataques de fuerza bruta por diccionario. A diferencia de `SHA-256` o `MD5`, que están optimizados para ser rápidos, `BCrypt` está diseñado para ser lento.

El proceso es **unidireccional**: la aplicación nunca puede recuperar la contraseña original, únicamente verificar si una candidata coincide. Esto elimina la posibilidad de exponerla en una respuesta HTTP, en logs o en un respaldo.

6.5 Mitigación 3 — Autenticación

Antes: la validación comparaba la contraseña recibida con la almacenada en texto plano, con una consulta que además era vulnerable a inyección.

Después: se recupera el usuario por nombre y se verifica la credencial con BCrypt.Verify, que recalcula el hash de la contraseña candidata usando el *salt* embebido en el hash almacenado y compara los resultados. Se devuelve un mensaje de error **genérico e idéntico** tanto si el usuario no existe como si la contraseña es incorrecta; esto impide la **enumeración de usuarios**, es decir, que un atacante deduzca qué cuentas existen a partir de las diferencias en la respuesta.

Como mejora adicional de defensa en profundidad se recomienda regenerar el identificador de sesión tras un inicio de sesión exitoso, limitar el número de intentos por cuenta y por dirección IP, y registrar los intentos fallidos.

6.6 Mitigación 4 — Cross-Site Scripting

Antes: @Html.Raw(comment) insertaba el contenido sin transformación alguna en el DOM.

Después: se codifica la salida antes de renderizarla, de modo que los caracteres estructurales del HTML se convierten en sus entidades correspondientes (< en <, > en >, " en "). El navegador recibe entonces una cadena de texto y la muestra tal cual, sin interpretarla como marcado: el payload queda visible como contenido literal y no se ejecuta.

Nota técnica (conviene incluirla, demuestra comprensión del framework): en Razor la sintaxis @comment ya aplica codificación HTML automática, por lo que basta con escribirla así. Usar @Html.Encode(comment) produce una **doble codificación** —Razor vuelve a codificar la cadena ya codificada—, con el resultado de que el usuario ve en pantalla <script> en lugar del texto original. La vulnerabilidad queda igualmente mitigada, pero la salida es incorrecta desde el punto de vista funcional. Si se requiere permitir HTML enriquecido de forma segura, la alternativa es un saneador con lista de permitidos, como HtmlSanitizer, y nunca Html.Raw sobre entrada de usuario.

Se complementa con una cabecera Content-Security-Policy que bloquee scripts en línea y con la marca HttpOnly en la cookie de sesión.

6.7 Mitigación 5 — IDOR

Antes: el endpoint recibía un id arbitrario y devolvía el registro correspondiente sin verificación alguna.

Después: se aplican tres controles encadenados:

1. **Verificación de autenticación:** se consulta el UserId de la sesión; si no existe, se responde 401 Unauthorized.
2. **Verificación de pertenencia:** se compara el id solicitado contra el identificador de la sesión; si no coinciden, se responde 403 Forbidden. La

decisión de autorización se toma con un valor del lado del servidor, no con un parámetro enviado por el cliente.

3. **Minimización de datos:** la respuesta se construye con un objeto anónimo que expone únicamente Id, Username y Email; los campos Balance y PasswordHash dejan de salir del servidor.

Se recomienda además eliminar por completo el endpoint GET /api/users o restringirlo a un rol administrativo mediante [Authorize(Roles = "Admin")], ya que ningún usuario final necesita el listado íntegro de cuentas.

Nota técnica: Forbid() invoca el esquema de autenticación configurado; en una aplicación que solo usa sesión, sin esquemas de autenticación registrados, esta llamada puede lanzar una excepción. Si eso ocurre en tu proyecto, sustitúyela por return StatusCode(403); o return Forbid() tras registrar un esquema de cookies. Documentar este ajuste suma en la evaluación.

6.8 Validación de las mitigaciones

# Prueba ejecutada	Resultado esperado	Resultado obtenido	Evidencia
1 Entrada SQL Injection en el buscador	No debe devolver todos los usuarios	(0 resultados)	Captura 15
2 Login con credencial insegura	No debe permitir acceso	(Credenciales inválidas)	Captura 19
3 Contenido XSS en comentarios	Debe mostrarse como texto	(texto plano, sin ejecución)	Captura 21
4 GET /api/user/{otro-id}	Unauthorized o Forbid	(403 Forbidden)	Captura 25
5 GET /api/user/{id} sin sesión	Unauthorized	(401 Unauthorized)	Captura 26
6 Respuesta del API	Sin Password PasswordHash	ni (campos ausentes)	Captura 24

6.9 Tabla comparativa final

Vulnerabilidad	OWASP 2021	Comportamiento antes	Control aplicado	Comportamiento después
SQL Injection	A03 Injection	— La entrada altera la lógica de la consulta y expone la tabla completa	Consultas LINQ parametrizadas por EF Core	La entrada se trata como dato literal
Contraseñas en texto plano	A02 Cryptographic Failures	— Credenciales legibles en base de datos y en respuestas del API	Hashing con Almacenamiento BCrypt (salt unidireccional, único + factor de trabajo)	imposible de revertir
Autenticación insegura	A07 Identification and Authentication Failures	— Comparación directa de cadenas, mensajes de error diferenciados	BCrypt.Verify y mensaje de error genérico	Acceso denegado sin filtrar la existencia de la cuenta
Cross-Site Scripting	A03 Injection	— El navegador ejecuta el script inyectado	Codificación de salida contextual y CSP	El contenido se muestra como texto inerte
IDOR	A01 Broken Access Control	— Cualquier cliente lee cualquier registro	Verificación de sesión, de pertenencia y DTO reducido	401 / 403 y respuesta con campos mínimos
Exposición masiva de datos	A01 Broken Access Control	— Listado íntegro de usuarios sin autenticación	Endpoint retirado o restringido por rol	Acceso limitado a personal autorizado

6.10 Evidencias solicitadas (P2F)

# Evidencia solicitada	Archivos
1 Capturas antes/después por vulnerabilidad	Capturas 14–15, 16–17, 18–19, 20–21, 23–25
2 Rama secure	Captura 13
3 Pruebas fallidas de los ataques previos	Capturas 15, 19, 21, 25, 26
4 Código corregido	Captura 27 — evidencias/P2F/27-codigo-corregido.png
5 Tabla comparativa final	Sección 6.9

7. Conclusiones

El ejercicio permite constatar que las cinco vulnerabilidades analizadas comparten una misma causa raíz: **la aplicación confía en datos que el cliente controla**. En la inyección SQL y en el XSS, la confianza se deposita en el contenido enviado por el usuario, que termina mezclándose con la estructura de una consulta o de un documento HTML. En el IDOR, la confianza se deposita en un identificador de la URL para decidir a qué recurso se concede acceso. En el manejo de contraseñas, el problema es de diseño: se almacena un secreto en una forma que permite recuperarlo.

De ahí se derivan cuatro principios aplicables a cualquier desarrollo:

1. **Separar datos de instrucciones**, mediante consultas parametrizadas y codificación de salida según el contexto de destino.
2. **Decidir la autorización siempre del lado del servidor**, comparando el recurso solicitado contra la identidad establecida en la sesión, nunca contra un parámetro del cliente.
3. **Minimizar la exposición de datos**, devolviendo objetos de transferencia diseñados para ese fin en lugar de entidades del modelo de dominio.
4. **Aplicar defensa en profundidad**, sin depender de un único control: hashing adaptativo, cabeceras de seguridad, limitación de tasa, registro de eventos y revisión de código.

Un hallazgo relevante del ejercicio es que la corrección de cada vulnerabilidad requirió modificaciones mínimas en términos de líneas de código, mientras que su

explotación resultaba trivial. El costo de programar de forma segura desde el inicio es sustancialmente menor que el de remediar y responder a un incidente.

8. Instrucciones de ejecución segura (para el README)

Clonar y ubicarse en la rama remediada

```
git clone <url-del-repositorio>
```

```
cd VulnerableApp
```

```
git checkout secure
```

Restaurar dependencias (incluye BCrypt.Net-Next)

```
dotnet restore
```

Aplicar migraciones

```
dotnet ef database update
```

Ejecutar

```
dotnet run
```

Advertencia: la rama main contiene código deliberadamente vulnerable con fines didácticos. No debe desplegarse en ningún entorno accesible desde red pública ni utilizarse como base de un proyecto real.

9. Referencias

- OWASP Top 10:2021 — A01 Broken Access Control, A02 Cryptographic Failures, A03 Injection, A07 Identification and Authentication Failures.
- OWASP Cheat Sheet Series: Cross Site Scripting Prevention, SQL Injection Prevention, Password Storage, Authorization.
- Microsoft Learn — Prevención de ataques XSS en ASP.NET Core; consultas SQL sin formato en EF Core.

Anexo A — Checklist de capturas

Antes de entregar, verifica que cada imagen sea legible, muestre la URL o el comando ejecutado y esté fechada por el sistema.

#	Archivo	Contenido	✓
0	00-entorno.png	Versión del SDK y app corriendo	☐
1	P2D/01-codigo-controller.png	CommentController.cs	☐

#	Archivo	Contenido	✓
2	P2D/02-codigo-vista.png	Index.cshtml con @Html.Raw	☐
3	P2D/03-comentario-legitimo.png	Comentario normal publicado	☐
4	P2D/04-payload-ingresado.png	Payload en el textarea	☐
5	P2D/05-alert-ejecutado.png	Alert desplegado	☐
6	P2D/06-devtools-dom.png	<script> en el DOM	☐
7	P2E/07-codigo-apicontroller.png	ApiController.cs	☐
8	P2E/08-get-user-1.png	Respuesta JSON usuario 1	☐
9	P2E/09-get-user-2.png	Respuesta JSON usuario 2	☐
10	P2E/10-get-user-3.png	Respuesta JSON usuario 3	☐
11	P2E/11-get-users.png	Listado completo	☐
12	P2E/12-sin-sesion.png	Petición sin sesión → 200 OK	☐
13	P2F/13-rama-secure.png	git branch / grafo de commits	☐
14	P2F/14-sqli-antes.png	Payload SQLi exitoso	☐
15	P2F/15-sqli-despues.png	Payload SQLi sin resultados	☐
16	P2F/16-bd-antes.png	Contraseñas en texto plano	☐
17	P2F/17-bd-despues.png	Hashes BCrypt	☐
18	P2F/18-login-antes.png	Bypass de login	☐
19	P2F/19-login-despues.png	Credenciales inválidas	☐
20	P2F/20-xss-antes.png	Alert ejecutándose	☐

# Archivo	Contenido	✓
21 P2F/21-xss-despues.png	Payload como texto	☐
22 P2F/22-xss-dom-despues.png	Entidades HTML en el DOM	☐
23 P2F/23-idor-antes.png	Acceso a usuario ajeno	☐
24 P2F/24-idor-propio.png	200 OK con campos reducidos	☐
25 P2F/25-idor-ajeno.png	403 Forbidden	☐
26 P2F/26-idor-sin-sesion.png	401 Unauthorized	☐
27 P2F/27-codigo-corregido.png	Diff o código de la rama secure	☐

Anexo B — Estructura de carpetas de entrega

/VulnerableApp

```

├── /evidencias
|   ├── /P2D (capturas 1–6)
|   ├── /P2E (capturas 7–12)
|   └── /P2F (capturas 13–27)
├── Reporte_Consolidado.pdf
└── README.md

```

Anexo C — Comandos de control de versiones

git add .

git commit -m "avance practica P2D - modulo de comentarios vulnerable"

git commit -m "avance practica P2E - api con IDOR"

git checkout -b secure

git commit -m "avance practica P2F - remediacion OWASP"